

Relatório de Adições e Ajustes do Trabalho (PPD)

Grupo: 10

Integrantes:

- Pedro de Mello Porto Daou (RA: 21010698)
- Enzo Garofalo Pampana (RA: 24008944)
- João Gabriel da Silva Leite (RA: 24757392)

Data da entrega final: 04/10/2026

1. Contexto

Durante a aula em que a atividade ocorreu, a base do projeto foi desenvolvida e versionada até o commit `f67924d`. Nesse estágio inicial, a estrutura dos programas sequencial e paralelo, a leitura de entrada com a classe `Teclado` e os algoritmos básicos de intercalação já estavam funcionais.

Este relatório descreve detalhadamente o que precisou ser acrescentado, testado e documentado após a aula para concluir os requisitos exigidos pelo trabalho.

2. O que foi acrescentado e finalizado

2.1. Robustez do Paralelismo e Sincronização

(`ParallelMergeSort.java`)

- **Cancelamento Cooperativo e Segurança com `join()`**: Aprimoramos o tratamento de interrupções e falhas. Caso uma thread falhe ou a thread principal seja interrompida, um sinal de cancelamento atômico é emitido e todas as worker threads ativas são aguardadas via `join()` antes de o método propagar a exceção. Isso garante que nenhuma thread continue escrevendo no vetor em segundo plano.
- **Tratamento de Exceções de Memória (`OutOfMemoryError`)**: Adicionamos tratamento explícito para falhas de alocação em vetores muito grandes ou

buffers auxiliares, exibindo mensagens orientativas para o usuário ajustar os parâmetros da JVM (`Xmx`).

2.2. Benchmarking Automatizado e Análise de Desempenho

- **Medição Precisa e Isolada:** Criamos uma rotina de benchmark que aplica aquecimento de JVM (*warmup*), alterna a ordem de execução entre sequencial e paralelo e calcula a **mediana de 5 amostras** para vetores de 100 mil, 1 milhão e 10 milhões de elementos.
- **Isolamento de I/O:** Garantimos que tempos de alocação, leitura do teclado e exibição fossem desconsiderados da medição do tempo puro de ordenação.
- **Resultados Obtidos:** Registramos ganho de desempenho significativo (*speedup* superior a 5x para vetores de milhões de elementos em uma máquina com 16 processadores).

2.3. Testes Automatizados Unitários (`src/test/java/`)

- Ampliamos a cobertura de testes em `ParallelMergeSortTest` para tratar cenários de borda:
 - Vetores vazios ou com apenas 1 elemento.
 - Vetores com números negativos e elementos duplicados.
 - Vetores com tamanhos e partições ímpares.
 - Execuções onde o número de threads ordenadoras supera a quantidade de elementos do vetor.
 - Validação cruzada com `Arrays.sort`.

2.4. Evidências de Execução e Capturas de Tela

- Geramos capturas de tela e registros de logs brutos do terminal cobrindo a execução de todas as aplicações exigidas:
 - `ProgramaSequencial` : ordenação sem uso de threads.
 - `ProgramaParalelo` : ordenação com `availableProcessors() - 1` ordenadoras e rodadas de juntadoras com logs ativados.
 - `Comparador` : execução comparativa na mesma entrada validando a igualdade dos resultados.

- **MaiorVetorAproximado** : estimativa do maior vetor suportado pela JVM com tratamento de estouro de memória.

2.5. Documentação Final e Diário de Bordo

- **Diário de Desenvolvimento:** Consolidamos o diário com a divisão de papéis de cada integrante, cronologia detalhada extraída do Git, impressões individuais e conclusão técnica.
 - **Relato de Testes:** Elaboramos o resumo sintético de 10 linhas sobre a suíte de testes e benchmarks.
 - **Roteiro de Apresentação:** Criamos um guia passo a passo para a demonstração do trabalho ao professor a partir de 06/10.
-